HarborGuard AI

Technical Build Specification for Coding Agent

MVP Development Brief

February 27, 2026

1 Executive Summary

Build HarborGuard AI: a web-based decision-support platform that helps mid-market electronics manufacturers respond to maritime supply chain disruptions. The system detects port disruptions, calculates financial exposure across inventory SKUs, simulates mitigation strategies, and recommends optimal actions with auto-generated execution drafts.

Core Value Proposition: Transform a 12-hour manual decision process into a 2-minute AI-assisted workflow that preserves millions in revenue.

2 System Overview

2.1 What We're Building

- **Single-tenant web application** (decision-support, not full autonomy)
- **Core workflow:** Disruption detected → Exposure mapped → Strategies simulated → Recommendation generated → Action drafts created
- **Target demo scenario:** Port of Los Angeles labor strike, 12-day delay, $38M revenue at risk

2.2 Explicitly Out of Scope

- Real-time streaming data pipelines (use mock/hardcoded data)
- Multi-tenant SaaS infrastructure
- Production ERP integrations (mock Oracle data)
- Machine learning models (use deterministic rules)
- Automated execution (human-in-the-loop only)
- Mobile applications

3 Technical Stack

Component	Technology
Backend API	Python + FastAPI
Database	PostgreSQL (local dev) / BigQuery (demo)
Frontend	React + TypeScript + Tailwind CSS
Optimization	Python (SciPy.optimize or PuLP)
LLM Integration	Google Gemini API
Deployment	Docker containers, Cloud Run or similar

4 Data Models

4.1 Core Entities

```
# Company (single tenant for MVP)
class Company:
    id: UUID
    name: str
    revenue_annual_millions: float # e.g., 420.0
    gross_margin_percent: float  # e.g., 0.28
    risk_tolerance: float        # 0.0-1.0, e.g., 0.3 (low tolerance)
    sla_weight: float            # 0.0-1.0, e.g., 0.9 (high priority)
    working_capital_limit: float # in USD, e.g., 5000000.0
    customer_churn_sensitivity: float # 0.0-1.0
    sla_target_percent: float    # e.g., 0.97

# Port
class Port:
    id: UUID
    code: str                    # "LA", "OAK", "SEA"
    name: str                    # "Port of Los Angeles"
    region: str                  # "US West Coast"

# SKU (Stock Keeping Unit)
class SKU:
    id: UUID
    sku_code: str                # "PCB-2024-X7"
    description: str
    daily_demand: int            # units per day
    unit_price: float            # USD
    unit_margin: float           # USD profit per unit
    criticality_score: float     # 0.0-1.0
    company_id: UUID

# Inventory Position
class Inventory:
    id: UUID
    sku_id: UUID
    on_hand_units: int
    reserved_units: int
    available_units: int         # computed: on_hand - reserved
```

```
# Shipment (Inbound)
class Shipment:
    id: UUID
    container_id: str
    sku_id: UUID
    port_id: UUID
    quantity: int
    original_arrival_date: date
    status: str                  # "in_transit", "delayed", "arrived"
    company_id: UUID

# Disruption Event
class Disruption:
    id: UUID
    port_id: UUID
    disruption_type: str         # "labor_strike", "weather", "congestion"
    severity_score: float        # 0.0-1.0
    expected_delay_days: int
    confidence_score: float      # 0.0-1.0
    detected_at: datetime
    is_active: bool

# Mitigation Strategy Template
class Strategy:
    id: UUID
    name: str                    # "Do Nothing", "Air Freight", "Split"
    description: str
    air_freight_percent: float   # 0.0-1.0
    reroute_percent: float       # 0.0-1.0
    buffer_stock_percent: float  # 0.0-1.0
    cost_multiplier_air: float   # e.g., 8.0x ocean cost
    cost_multiplier_reroute: float # e.g., 1.3x ocean cost
    holding_cost_per_unit_per_day: float
```

4.2 Calculation Results

```
# Exposure Analysis Result
class ExposureResult:
    disruption_id: UUID
    affected_shipments: List[Shipment]
    affected_skus: List[SKU]
    total_revenue_at_risk: float
    total_margin_at_risk: float
    inventory_runway_days: Dict[UUID, int] # sku_id -> days
    stockout_date: Dict[UUID, date]     # sku_id -> date

# Strategy Simulation Result
class StrategyResult:
    strategy_id: UUID
    disruption_id: UUID
    revenue_preserved: float
    mitigation_cost: float
    sla_achieved: float
```

```
    sla_penalty_cost: float
    net_financial_impact: float         # revenue_preserved - costs
    working_capital_required: float
    is_feasible: bool                   # meets working_capital_limit

# Final Recommendation
class Recommendation:
    disruption_id: UUID
    selected_strategy_id: UUID
    confidence_score: float
    reasoning: str                      # explainable AI output
    generated_email_supplier: str
    generated_email_logistics: str
    generated_executive_summary: str
    requires_approval: bool             # always True for MVP
    approved_by: Optional[str]
    approved_at: Optional[datetime]
```

5 API Specification

5.1 Endpoints to Implement

```
POST  /api/v1/disruptions
      Body: {port_code, type, delay_days, severity}
      -> Creates disruption, triggers analysis pipeline
      -> Returns: disruption_id

GET   /api/v1/disruptions/{id}/exposure
      -> Returns: ExposureResult JSON

GET   /api/v1/disruptions/{id}/strategies
      -> Returns: List[StrategyResult] with comparison

GET   /api/v1/disruptions/{id}/recommendation
      -> Returns: Recommendation (top strategy selected)

POST  /api/v1/recommendations/{id}/approve
      Body: {approver_name, notes}
      -> Marks recommendation as approved

GET   /api/v1/dashboard
      -> Returns: Current active disruption + summary stats

POST  /api/v1/seed-data
      -> Populates mock data for demo scenario
```

6 Core Algorithms

6.1 1. Inventory Runway Calculation

For each SKU affected by disruption at port p:

$$\text{Runway}_i = \frac{\text{Inventory}_i}{\text{DailyDemand}_i} \tag{1}$$

$$\text{AdjustedArrival}_{i,p} = \text{OriginalArrival}_i + \text{Delay}_p \tag{2}$$

$$\text{StockoutRisk}_i = \begin{cases} \text{true} & \text{if AdjustedArrival}_{i,p} > \text{Runway}_i \\ \text{false} & \text{otherwise} \end{cases} \tag{3}$$

6.2 2. Revenue at Risk

$$\text{RevenueAtRisk}_i = \text{DailyDemand}_i \times \text{UnitPrice}_i \times \text{DelayGap}_i \tag{4}$$

Where $\text{DelayGap}_i = \max(0, \text{AdjustedArrival}_{i,p} - \text{Runway}_i)$

$$\text{TotalRevenueAtRisk} = \sum_{i \in \text{AffectedSKUs}} \text{RevenueAtRisk}_i \tag{5}$$

6.3 3. Strategy Optimization

For each strategy $s \in S$:

$$R_s = \sum_{i \in \text{SKUs}} (\text{DailyDemand}_i \times \text{UnitMargin}_i \times \text{Availability}_{i,s} \times \text{DelayGap}_i) \tag{6}$$

$$C_s = \text{AirCost}_s + \text{RerouteCost}_s + \text{BufferCost}_s \tag{7}$$

$$P_s = \lambda \times \max(0, \text{SLA}_{\text{target}} - \text{SLA}_{\text{achieved},s}) \tag{8}$$

Where λ is penalty weight from Risk DNA.

$$\text{NetImpact}_s = \alpha R_s - \beta C_s - \delta P_s \tag{9}$$

Subject to:

$$C_s \leq \gamma \quad \text{(Working capital constraint)} \tag{10}$$

Where:

- α = risk tolerance coefficient (higher = more aggressive)
- β = cost sensitivity weight
- δ = SLA penalty weight
- γ = working capital limit

Optimal strategy:

$$s^* = \arg\max_{s \in S} \text{NetImpact}_s \quad \text{s.t.} \quad C_s \leq \gamma \tag{11}$$

6.4 4. Predefined Strategies

Strategy	Air Freight %	Reroute %	Buffer %
S_1: Do Nothing	0%	0%	0%
S_2: Full Air Freight	100%	0%	0%
S_3: Port Reroute (Oakland)	0%	100%	0%
S_4: Split (Recommended)	20%	60%	20%

Cost assumptions:

- Ocean freight baseline: $3,000 per container
- Air freight: 8x ocean cost = $24,000 per container
- Reroute to Oakland: 1.3x ocean cost = $3,900 per container
- Buffer stock holding: $0.50 per unit per day

7 LLM Integration (Gemini API)

7.1 Prompt Template for Action Generation

```
You are a VP of Supply Chain at a $420M electronics manufacturer.
A disruption has occurred with the following details:
- Port: Port of Los Angeles
- Type: Labor Strike
- Expected Delay: 12 days
- Affected SKUs: 18 high-priority items
- Revenue at Risk: $38.4M

Company Risk Profile:
- Risk Tolerance: Low (0.3)
- SLA Target: 97%
- Working Capital Available: $5M

OPTIMAL STRATEGY SELECTED:
- Reroute 60% via Port of Oakland
- Air freight 20% (most critical SKUs)
- Build buffer stock for 20%

Generate the following:
1. Professional email to Taiwanese supplier requesting reroute
2. Logistics rebooking request to freight forwarder
3. Executive summary memo for COO (3 bullet points max)

Be concise, professional, and specific about actions required.
```

7.2 Expected Outputs

Store in `Recommendation` object:

- `generated_email_supplier`: Full email text

- `generated_email_logistics`: Freight request text
- `generated_executive_summary`: Brief memo

8 Frontend UI Specification

8.1 Required Pages

8.1.1 1. Dashboard (Main View)

- **Header:** HarborGuard AI logo, company name
- **Alert Panel:** Large red/yellow banner if active disruption
- **Key Metric Cards:**
  - Revenue at Risk: $__M (large, red if ¿$10M)
  - Affected SKUs: __ items
  - Inventory Runway: __ days average
- **Action Button:** "View Recommendation" (primary CTA)

8.1.2 2. Disruption Detail Page

- Disruption summary card (port, type, severity, delay)
- Exposure breakdown table (SKU, quantity, runway, risk level)
- Strategy comparison table (4 rows, sortable by net impact)
- Recommended strategy highlight box (green border)
- "Approve Strategy" button (triggers approval workflow)

8.1.3 3. Recommendation Page

- Strategy summary with financial impact
- "Why this decision?" expandable section (explainability)
- Three generated draft emails in tabs
- Copy-to-clipboard buttons for each draft
- Approval workflow: Name input + Approve button

8.2 UI Design Principles

- **Minimalist:** One primary action per screen
- **Data-dense but scannable:** Tables for SKUs, cards for metrics
- **Color coding:** Red (critical), Yellow (warning), Green (optimal)
- **No charts for MVP:** Numbers and tables only

9 Demo Data Setup

9.1 Mock Company Profile

```
Company: "Apex Electronics Inc."
Revenue: $420M annually
Gross Margin: 28%
Location: Austin, TX
Imports: 60% from Asia (Taiwan, South Korea, Malaysia)
Primary Port: Los Angeles (45% of inbound)
SLA Commitment: 97% on-time to top 3 customers
Inventory Runway: 21 days on critical chips
Risk Tolerance: Low (conservative financial posture)
Working Capital Buffer: $5M for disruption response
```

9.2 Mock Disruption Scenario

```
Event: Labor Strike at Port of Los Angeles
Announced: Today
Expected Duration: 10-14 days
Capacity Reduction: 35%
Affected Shipments: 47 containers
Affected SKUs: 18 high-priority items
Immediate Revenue at Risk: $38.4M (45-day window)
```

9.3 Seed Data Command

Implement `POST /api/v1/seed-data` that populates:

- 1 Company (Apex Electronics profile)
- 3 Ports (LA, Oakland, Seattle)
- 25 SKUs (various PCB components, chips)
- 50 Shipments (distributed across 30 days)
- 1 Active Disruption (LA strike scenario)

10 Build Phases

- Set up FastAPI + PostgreSQL + React project structure
- Create database schema and models
- Implement seed data endpoint
- Build inventory runway calculator
- Implement exposure mapping logic
- Create revenue-at-risk computation

- Implement 4 strategy definitions
- Build constraint solver (SciPy or simple enumeration)
- Create strategy comparison endpoint
- Set up Gemini API client
- Build prompt templates
- Implement action generation endpoint
- Build React components (Dashboard, Tables, Cards)
- Connect to backend APIs
- Style with Tailwind CSS
- Disruption detail page
- Strategy comparison view
- Recommendation and approval flow
- End-to-end testing
- Demo script rehearsal data
- Bug fixes and polish

11 Success Criteria

The MVP is complete when:

1. **Functional:** User can trigger disruption → see exposure → view strategies → get recommendation with drafts
2. **Mathematically Correct:** Revenue at risk and strategy NPV calculations match expected demo values ($38.4M risk, $9.5M preserved with split strategy)
3. **Demo-Ready:** 5-minute walkthrough flows smoothly without manual data entry
4. **Explainable:** "Why this decision?" shows clear reasoning tied to Risk DNA parameters

12 File Structure

```
harborguard-ai/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py            # FastAPI entry point
│   │   ├── models.py          # SQLAlchemy models
│   │   ├── schemas.py         # Pydantic schemas
│   │   ├── routers/
│   │   │   ├── disruptions.py
│   │   │   ├── strategies.py
│   │   │   └── dashboard.py
│   │   ├── services/
│   │   │   ├── exposure.py    # Runway calculations
│   │   │   ├── optimizer.py   # Strategy optimization
│   │   │   └── llm_generator.py # Gemini integration
│   │   └── core/
│   │       ├── config.py
│   │       └── database.py
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── DisruptionAlert.tsx
│   │   │   ├── ExposureTable.tsx
│   │   │   ├── StrategyComparison.tsx
│   │   │   └── RecommendationPanel.tsx
│   │   ├── pages/
│   │   │   ├── DashboardPage.tsx
│   │   │   ├── DisruptionPage.tsx
│   │   │   └── RecommendationPage.tsx
│   │   ├── api/
│   │   │   └── client.ts
│   │   └── App.tsx
│   ├── Dockerfile
│   └── package.json
├── docker-compose.yml
└── README.md
```

13 Environment Variables

```
# Backend (.env)
DATABASE_URL=postgresql://user:pass@db:5432/harborguard
GEMINI_API_KEY=your_google_gemini_key_here
CORS_ORIGINS=http://localhost:3000

# Frontend (.env)
REACT_APP_API_URL=http://localhost:8000/api/v1
```

14 Dependencies

14.1 Backend (requirements.txt)

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
sqlalchemy==2.0.23
psycopg2-binary==2.9.9
pydantic==2.5.2
python-dotenv==1.0.0
httpx==0.25.2
scipy==1.11.4
numpy==1.26.2
google-generativeai==0.3.1
```

14.2 Frontend (package.json key deps)

```
{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0",
    "axios": "^1.6.2",
    "tailwindcss": "^3.3.0",
    "lucide-react": "^0.294.0",
    "recharts": "^2.10.0"
  }
}
```